Divide and Conquer: Closest Pair of Points

Lecture Notes Transcribed

1 Introduction to the Problem

The problem that we are going to discuss now comes from computational geometry. We will use an algorithm design strategy here.

The Problem Statement:

- **Input:** A set of points in the plane.

- **Output:** The closest pair of points.

The points are represented by their x and y coordinates. We are interested in finding a pair of points which are closest to each other among all possible pairs of points in the plane.

Given any two points (x_1, y_1) and (x_2, y_2), the distance is calculated using the usual Euclidean distance formula:

$$\text{Distance} = \sqrt{(x_1 - x_2)^2 + (y_1 - y_2)^2}$$

For a given set of n points, there are $\binom{n}{2}$ pairs of points, and therefore $\binom{n}{2}$ distances. We want to pick the minimum distance.

1.1 The Naive Approach

The obvious way to do it is to compute the distance between every pair of points and find the minimum. Since there are $\binom{n}{2}$ distances, the algorithm takes $O(n^2)$ time.

Now the obvious question is, can we do better? Do we really need to compute all $\binom{n}{2}$ distances? Well, that is an interesting question, and the answer is No. We will do it much faster than $O(n^2)$. Somehow, we will avoid computing all $\binom{n}{2}$ distances; that is the moral of the story.

2 Thinking Inductively

Let us now see how we go about designing the algorithm. You might be wondering why we need to compute every pair of distances. Can we look at the geometry of the points and come up with an idea that avoids the computation of $\binom{n}{2}$ distances?

Think about an inductive approach: suppose I remove one point from the set, find the closest pair among the rest, and then put the point back. Can we get rid of checking some points? This is something we need to think about when we have a new problem to solve.

Let us look at this approach (although this will not be the final approach we use). Suppose you remove a point u from the region. Somehow, you find the minimum distance among the remaining points. Now, when you put the point u back into the region, what will be the new minimum distance? The obvious thing is to compute the distance from all other points to u to check if there is a new minimum distance.

But this is not an efficient way because there are $n - 1$ other points in the region, and we would need to compute $n - 1$ new distances. The recurrence relation would lead to an $O(n^2)$ solution:

$$T(n) = (n - 1) + T(n - 1) \implies O(n^2)$$

Can we avoid computing all $n-1$ distances when we put the point u back? This is a crucial observation: Suppose the minimum distance in the region (except point u) is recursively found to be δ. Any point that is in contention for a new minimum distance must lie within a circle of radius δ around point u. Other points are too far away. If you can quickly compute the distances to only those points inside the circle, you have achieved something. If we can cut the $n-1$ computations down to something faster, perhaps $O(\log n)$ or constant time, we are in good shape.

3 The One-Dimensional Case

Before we go further, let us solve this problem in one dimension. You are given points on a single line, and you want to find the closest pair.

How will you do this? We do not need to compute all $\binom{n}{2}$ distances. The important thing to notice here is that for any particular point, the only candidate points are its adjacent points.

1D Algorithm:

1. Sort the points by reading their coordinates from left to right.
2. Check the distance between each adjacent pair.
3. Choose the minimum distance.

How much time does this algorithm take? Sorting takes $O(n \log n)$ time. We started with $\binom{n}{2}$ distances but reduced the time complexity to $O(n \log n)$. Sorting dominates this procedure.

Whenever you are given a problem in two dimensions, it always pays to check what the problem means in one dimension. You learn at least that it is possible to do it in less than $O(n^2)$ time.

4 Divide and Conquer in 1D

Is there something like sorting that we can do which ultimately helps us in two dimensions? Let's apply a divide and conquer approach. We divide the points into two parts: the left part (L) and the right part (R).

We recurse to find the minimum distance on both halves. Let δ_L and δ_R be the respective minimum distances on the left and right. The overall minimum distance δ computed so far is $\min(\delta_L, \delta_R)$.

The only thing we have not checked is the distance between points on the left and points on the right. Specifically, we only need to check the distance between the *last* point on the left and the *first* point on the right. If this distance is less than δ, it becomes our new minimum.

Finding the median point to divide the array takes linear time $O(n)$. The recurrence looks like:

$$T(n) = 2T(n/2) + O(n)$$

And we know this solves to $O(n \log n)$.

5 Divide and Conquer in 2D

Now let us do this on the plane. We divide the point set into two parts based on their x-coordinates. The leftmost $n/2$ points form one partition (L), and the rightmost $n/2$ points form the other partition (R).

Recursively find the minimum distance in the left half (δ_L) and the right half (δ_R). Let:

$$\delta = \min(\delta_L, \delta_R)$$

An important observation is that if there is a candidate pair with a distance less than δ, one point must be on the left half and the other must be on the right half.

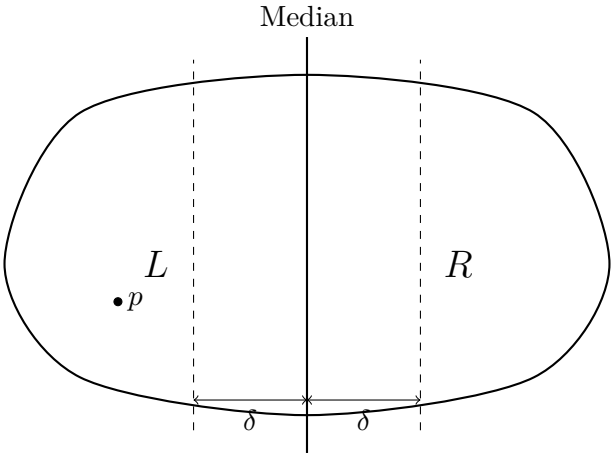

Figure 1: Dividing the plane using the median line and identifying the δ band.

The recurrence can be written as:

$$T(n) = 2T(n/2) + \text{Combine Step}$$

If the combine step takes $O(n)$ time, then the overall recurrence will yield $O(n \log n)$, putting us in good shape.

The natural but naive way of combining is to pick every point on the left and compute its distance to every point on the right. That gives $\frac{n}{2} \times \frac{n}{2} = \frac{n^2}{4}$ distances, which drags the time complexity back to $O(n^2)$. We need a smarter trick to speed things up.

5.1 The Sparsity Condition

Notice where candidate points lie. The candidate points must be within a distance of δ from the median line. If a point lies outside this region, its distance to any point on the opposite side would be greater than δ. So we can ignore all points outside the δ-band.

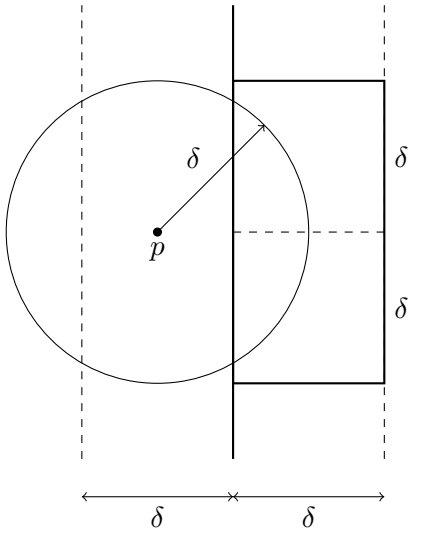

Figure 2: Magnified view of the boundary region. The sparse distribution ensures few points fall within the candidate box.

If we take any point on the left boundary and draw a circle of radius δ around it, there cannot be many points clustered inside the circle because the minimum distance between any two points within the left half is already strictly $\geq \delta$. The points are well-spread.

For a specific point on the left, candidate pairs on the right must lie within a $\delta \times 2\delta$ bounding rectangle. Due to the sparsity property, there can only be a small, constant number of points in this region (at most 6).

6 The Final Algorithm

1. **Initialization Step:** Sort the points based on their x-coordinates and y-coordinates. We have two arrays: one sorted by x and one sorted by y. This initialization is done before the recursion starts, and these arrays will be passed to each recursive step.

2. **Divide Step:** Let the value of the median on the x-coordinate be X_{med}. Split the points into X_L and X_R. Points in X_L have x-coordinates $\leq X_{med}$, and the other points are in X_R. Split the Y array into Y_L and Y_R. Y_L contains the same points as X_L but sorted on y-coordinates. Similarly for Y_R.

3. **Recursive Step:** Recursively find:

   - d_L: Distance of the closest pair in (X_L, Y_L).
   - d_R: Distance of the closest pair in (X_R, Y_R).

   Let $d = \min(d_L, d_R)$.

4. **Combine Step:**

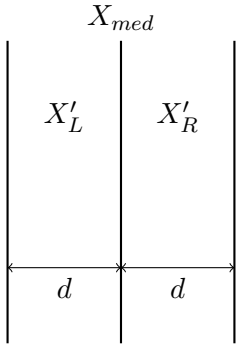

Figure 3: Filtering the points within distance d of the median.

Filter the points that fall within the boundary:

- X'_L: points from X_L whose x-coordinate is $\geq X_{med} - d$.
- Y'_L: points from X'_L sorted by y-coordinate.
- X'_R: points from X_R whose x-coordinate is $\leq X_{med} + d$.
- Y'_R: points from X'_R sorted by y-coordinate.

Maintain these points as (x', y'). For each point in Y'_L, scan Y'_R to find candidate points whose y-coordinates fall in the range $[y' - d, y' + d]$. Due to the sparsity property, we will only need to check at most 6 points.

Compute the distance for these candidate pairs. Choose the minimum between d and any newly discovered closer distance.

6.1 Analysis

- **Sorting:** $O(n \log n)$ initially.
- **Two Recursive Calls:** $2T(n/2)$.
- **Combine Step:** Linear scan taking $O(n)$ time.

Thus, the recurrence relation is $T(n) = 2T(n/2) + O(n)$, which resolves to an overall time complexity of $O(n \log n)$.